ADVANCED OBJECT-ORIENTED PROGRAMMING USING JAVA

CO-1: TEAM-BASED CASE STUDY EXERCISES

CASE STUDY 5

USER ACCOUNTS, EMAIL VERIFICATION AND CONTACT DETAILS

Name	V. Gautam
Register Number	2520030316
Class / Section	Section-1 cluster 1

Based on the Case Study 5 questions in the supplied CO-1 assignment.

CASE STUDY 5 – PROBLEM STATEMENT

A Software-as-a-Service (SaaS) company wants to develop a user account management system. The application should support user registration, email verification, optional phone numbers, postal addresses, account activation, suspension and deletion. The existing system has problems such as incorrectly formatted email addresses, verified users without verification dates, expired verification tokens being accepted, inconsistent phone formats, incomplete postal addresses, suspended users being allowed to log in, and duplicate email addresses with different capitalization.
The objective is to redesign the application using strong object-oriented domain modelling principles.

Q1. Identification of Invariants (6 Marks)

Domain invariants are business rules that must always be true in the user account management system.
1. Valid email address: Every email address must follow a valid format. This prevents incorrect registration details and failed email delivery.
2. Verification date: A verified email must have a recorded verification date. Otherwise, the system may show an incorrect verification status.
3. Token expiry: A verification token must match and must not be expired. This prevents old or misused tokens from verifying an account.
4. Phone number format: An optional phone number must follow a standard format. This avoids inconsistent or unusable contact numbers.
5. Complete postal address: If an address is provided, required fields such as street, city, postal code and country must be present. This prevents incomplete address records.
6. Account status: Suspended and deleted users must not be allowed to log in. This protects the application from unauthorized access.
7. Unique email: The normalized email address must be unique across all users. This prevents duplicate accounts caused by different capitalization.
Conclusion: These invariants protect user information, maintain correct account states and improve application security.

Q2. Entities and Value Objects (6 Marks)

An entity has a unique identity and may change during its lifetime. A value object is identified by its value and is usually immutable.

Concept	Classification	Justification
User Account	Entity	Has a unique user ID. Its email and account status may change.
User Identifier	Value Object	Represents the user's identity value and should not change.
Email Address	Value Object	Compared by its normalized email value.
Phone Number	Value Object	Represents a validated phone number.
Postal Address	Value Object	Contains street, city, postal code and country values.
Verification Token	Value Object	Contains a token value and expiry time; it is replaced when a new token is generated.

Conclusion: The User Account is tracked by its identity, while contact details and verification data are represented as value objects.

Q3. Making Illegal States Unrepresentable (8 Marks)

A self-enforcing object validates its own data and checks business rules before allowing an operation.

Problems with primitive fields and Boolean flags

- An invalid email address may be stored as a normal String.
- emailVerified may be true while verifiedAt is null.
- An expired verification token may still be accepted if expiry is not checked.
- Phone numbers may be stored in different formats.
- Separate nullable address fields may create incomplete addresses.
- Several Boolean flags can represent contradictory account states.
- Suspended or deleted users may be allowed to log in if status is not checked.

Improved design

Domain Type	Purpose
EmailAddress	Validates the email format and stores it in lowercase.
PhoneNumber	Validates and normalizes an optional phone number.
PostalAddress	Accepts an address only when all required parts are present.
VerificationToken	Stores a random token and its expiry time.
EmailVerification	Represents Pending or Verified state without contradictory flags.
AccountStatus	Uses ACTIVE, SUSPENDED and DELETED states.
UserAccount	Controls registration, verification, suspension and login.

Java Program

```
import java.time.LocalDateTime;
import java.util.*;
import java.security.SecureRandom;

class EmailAddress {
    private final String value;
    EmailAddress(String email) {
        if (email == null ||
            !email.trim().matches("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$"))
            throw new IllegalArgumentException("Invalid email address");
        value = email.trim().toLowerCase(Locale.ROOT);
    }
    String value() { return value; }
}

class PhoneNumber {
    private final String value;
    PhoneNumber(String number) {
        String n = number.replaceAll("[\\s-]", "");
        if (!n.matches("\\+?[0-9]{10,15}"))
            throw new IllegalArgumentException("Invalid phone number");
        value = n;
    }
}

class PostalAddress {
    private final String street, city, postalCode, country;
    PostalAddress(String street, String city, String postalCode, String country) {
        if (blank(street) || blank(city) || blank(postalCode) || blank(country))
            throw new IllegalArgumentException("Incomplete postal address");
        this.street = street; this.city = city;
        this.postalCode = postalCode; this.country = country;
    }
    private boolean blank(String s) { return s == null || s.trim().isEmpty(); }
```

```
}

class VerificationToken {
    private final String value;
    private final LocalDateTime expiry;
    VerificationToken() {
        String chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
        SecureRandom random = new SecureRandom();
        StringBuilder b = new StringBuilder();
        for (int i = 0; i < 6; i++)
            b.append(chars.charAt(random.nextInt(chars.length())));
        value = b.toString();
        expiry = LocalDateTime.now().plusMinutes(10);
    }
    boolean isValid(String entered) {
        return value.equals(entered) && LocalDateTime.now().isBefore(expiry);
    }
    String value() { return value; }
}

class EmailVerification {
    private final VerificationToken token;
    private final LocalDateTime verifiedAt;

    private EmailVerification(VerificationToken t, LocalDateTime date) {
        token = t; verifiedAt = date;
    }
    static EmailVerification pending() {
        return new EmailVerification(new VerificationToken(), null);
    }
    static EmailVerification verified() {
        return new EmailVerification(null, LocalDateTime.now());
    }
    EmailVerification verify(String entered) {
        if (token == null || !token.isValid(entered))
            throw new IllegalArgumentException("Invalid or expired token");
        return verified();
    }
    boolean isVerified() { return verifiedAt != null; }
    String tokenValue() { return token == null ? null : token.value(); }
}

enum AccountStatus { ACTIVE, SUSPENDED, DELETED }

class UserAccount {
    private final String userId;
    private final EmailAddress email;
    private final Optional<PhoneNumber> phone;
    private final Optional<PostalAddress> address;
    private EmailVerification verification;
    private AccountStatus status;

    UserAccount(String id, EmailAddress email,
                Optional<PhoneNumber> phone,
                Optional<PostalAddress> address) {
        userId = id; this.email = email;
        this.phone = phone; this.address = address;
        verification = EmailVerification.pending();
        status = AccountStatus.ACTIVE;
    }
    void verifyEmail(String token) {
        verification = verification.verify(token);
    }
    void login() {
        if (status != AccountStatus.ACTIVE)
            throw new IllegalStateException("Account is not active");
        if (!verification.isVerified())
```

```
            throw new IllegalStateException("Email is not verified");
        System.out.println("Login successful!");
    }
    void suspend() { status = AccountStatus.SUSPENDED; }
    void delete() { status = AccountStatus.DELETED; }
    String id() { return userId; }
    String email() { return email.value(); }
    String token() { return verification.tokenValue(); }
}

class UserRepository {
    private final Map<String, UserAccount> users = new HashMap<>();

    synchronized void register(UserAccount user) {
        for (UserAccount u : users.values()) {
            if (u.email().equals(user.email()))
                throw new IllegalArgumentException("Email already registered");
        }
        users.put(user.id(), user);
    }
}

public class CaseStudy5Demo {
    public static void main(String[] args) {
        UserRepository repository = new UserRepository();

        EmailAddress email = new EmailAddress("sameer@gmail.com");
        UserAccount user = new UserAccount(
            "U101", email,
            Optional.of(new PhoneNumber("+919876543210")),
            Optional.empty()
        );

        repository.register(user);
        System.out.println("Registration successful!");
        System.out.println("User ID: " + user.id());
        System.out.println("Email: " + user.email());

        String token = user.token();
        System.out.println("Verification Token: " + token);

        user.verifyEmail(token);
        System.out.println("Email verified successfully!");
        user.login();

        user.suspend();
        try {
            user.login();
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

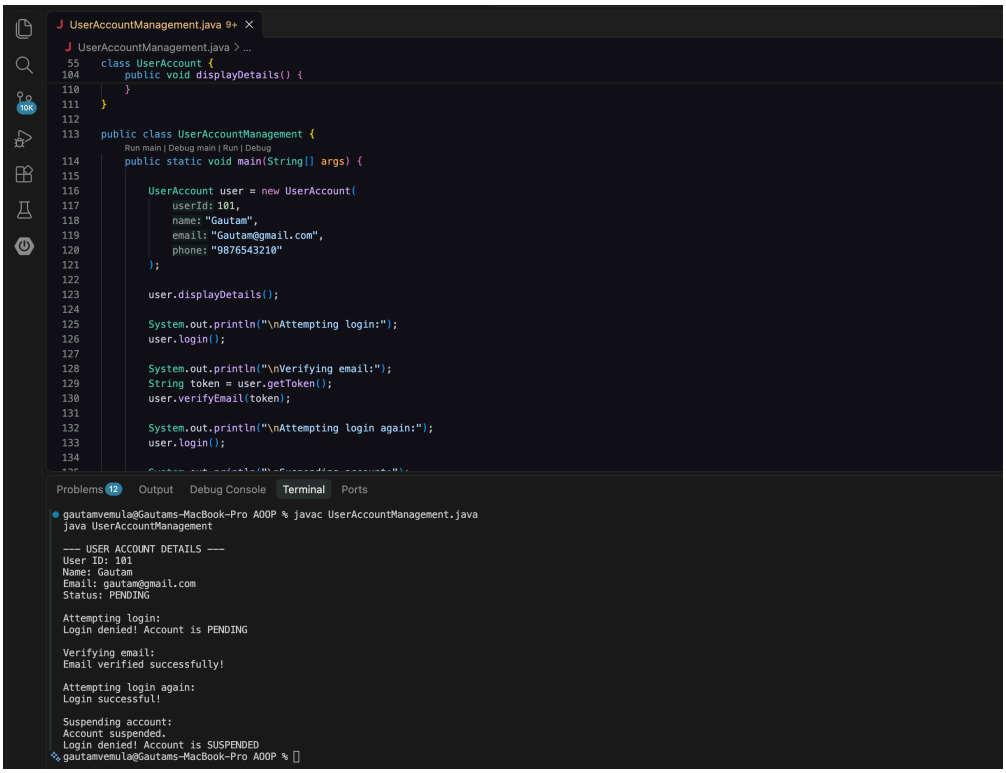
Program Explanation

- EmailAddress validates the email format and converts it to lowercase.
- PhoneNumber validates a phone number. Phone and postal address are optional in UserAccount.
- PostalAddress rejects an address if any required address field is empty.
- VerificationToken generates a random six-character token and gives it a ten-minute expiry time.
- EmailVerification changes from Pending to Verified only when the token is correct and not expired.
- UserAccount keeps its fields private and checks verification and account status before login.
- UserRepository checks for an existing normalized email before registering a user.

Expected Output

Registration successful!
User ID: U101
Email: sameer@gmail.com
Verification Token: ABC123 (random)
Email verified successfully!
Login successful!
Account is not active
The token shown in the actual output will be randomly generated. ABC123 is only an example.

Paste a screenshot of successful Java execution here



How the Program Prevents Errors

- Rejects incorrectly formatted email addresses.
- Normalizes email addresses to lowercase.
- Rejects invalid phone numbers and incomplete addresses.
- Rejects incorrect or expired verification tokens.
- Does not allow login before email verification.
- Does not allow suspended or deleted accounts to log in.
- Uses private fields and controlled methods to protect account data.

Note: This is a teaching example. The repository is stored in memory and its duplicate check is suitable only for a single running program. A real application must enforce email uniqueness with a database UNIQUE constraint, securely store verification tokens and send them through an email service.

Q4. Email Verification and Account Lifecycle (6 Marks)

The registration workflow must record the user ID, normalized email address, optional phone number, optional postal address, account creation time, verification state and account status.

1. Registration and verification workflow

10. Collect the user's details and validate the email address.
11. Check whether the normalized email address is already registered.
12. Create the account in Pending email-verification state.
13. Generate a random verification token and set an expiry time.

14. Send the token or verification link to the user's email address.
15. When the user submits the token, check that it matches and has not expired.
16. If valid, change the email verification state to Verified and record the verification date.

2. Handling expired tokens

Reject the expired token, show an error message, generate a new token, set a new expiry time and send a new verification email.

3. When a verified user changes their email

Validate the new email, check its uniqueness, mark the new email as Pending, generate a new token and send a verification email. The new email becomes verified only after successful verification.

4. Preventing suspended and deleted users from logging in

Account Status	Login Rule
ACTIVE	Allow login only when the email is verified.
SUSPENDED	Reject login.
DELETED	Reject login.

5. Preventing contradictory verification states

Use a dedicated EmailVerification type instead of separate Boolean and nullable fields. Pending contains a token and expiry time. Verified contains a verification date. This avoids invalid combinations such as emailVerified = true with verifiedAt = null.
Conclusion: A controlled registration and verification workflow protects user accounts and prevents unauthorized login.

Workflow Diagram

Step	Action	Result
1	User registration	User details are collected.
2	Email validation	Invalid email is rejected.
3	Token generation	Pending verification is created.
4	Token validation	Token and expiry are checked.
5	Successful verification	Email is marked Verified.
6	Login check	Only active, verified users can log in.

Q5. Cross-Aggregate Rules and Design Evaluation (4 Marks)

1. Is email uniqueness an invariant of an individual User object?

No. Email uniqueness is not an invariant of a single User object because it depends on every registered user. One user object cannot independently know whether another user is using the same email. Therefore, email uniqueness is a cross-aggregate rule.

2. How should the application enforce email uniqueness?

17. Normalize the email address by converting it to lowercase.
18. Check whether the normalized email already exists.
19. Reject registration if the email is already used.
20. Store the normalized email and enforce uniqueness in the database.

Java example:

```
String email = "SAMEER@gmail.com";
email = email.toLowerCase(Locale.ROOT);
System.out.println(email);
Output:
sameer@gmail.com
```

3. What happens if two users register with the same email simultaneously?

Both requests may check the database at nearly the same time and find that the email is available. Both may then attempt registration. This can create duplicate accounts. This problem is called a Race Condition.

4. Additional database protection

The database must enforce a UNIQUE constraint on the normalized email address. This is essential because an application-level check alone cannot prevent simultaneous requests from inserting duplicates.

SQL example:

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    email VARCHAR(100) NOT NULL UNIQUE,
    status VARCHAR(20)
);
```

For case-insensitive uniqueness in PostgreSQL, a unique index can be created:

```
CREATE UNIQUE INDEX unique_email
ON Users (LOWER(email));
```

5. Advantage and disadvantage of strongly typed verification

Advantage	Disadvantage
Prevents contradictory verification states.	Requires additional classes and code.
Improves data consistency and validation.	Increases initial design complexity.
Makes the model easier to understand and maintain.	Developers must understand the extra domain types.

Conclusion: Email uniqueness must be checked across users and protected by a database constraint to prevent duplicate registrations.

QUICK REVISION – IMPORTANT TERMS

Term	Simple Meaning
Invariant	A business rule that must always be true.
Entity	An object with a unique identity.
Value Object	An object identified by its values.
Domain-specific type	A class representing a meaningful business concept.
Verification Token	A temporary code used to verify an email.
Race Condition	A problem caused by simultaneous operations.
Cross-aggregate rule	A rule that depends on more than one entity or aggregate.
Unique Constraint	A database rule that prevents duplicate values.

END OF CASE STUDY 5